CYCLOPS: Deep Learning Based Monocular Vision Obstacle Detection and Avoidance for Autonomous Quadrotor UAV Navigation

Project Implementation Report

Name: Amit Singh Chauhan

Registration No.: 23BAI1450

Course: BCSE428L – Autonomous Drones

1. Project Links

Source code (GitHub)	https://github.com/ac265640/CYCLOPS
Project website (live)	https://ac265640.github.io/CYCLOPS/
Demo videos	https://ac265640.github.io/CYCLOPS/#demo
This report (PDF)	https://ac265640.github.io/CYCLOPS/report/CYCLOPS_Project_Report.pdf
Raw results (CSV / JSON)	https://github.com/ac265640/CYCLOPS/tree/main/results
Syllabus mapping	https://github.com/ac265640/CYCLOPS/blob/main/docs/curriculum_mapping.md
ROS 2 package	https://github.com/ac265640/CYCLOPS/tree/main/ros2_ws/src/cyclops_ros
Reference paper (IEEE)	https://doi.org/10.1109/ICUAS48674.2020.9213901

2. Abstract

This report describes the implementation of **CYCLOPS** (*Camera-only, Yaw-aware Collision avoidance with Learned Obstacle Perception for Small UAVs*), the project proposed in the initial report. A simulated quadrotor carries one forward-facing camera, with no LiDAR and no stereo. A lightweight convolutional detector finds obstacles in each frame, and a monocular depth network estimates their distance. An extended Kalman filter tracks every obstacle, using the reference paper's range-and-bearing (Mode 1) or bearing-only (Mode 2) localisation. A field-of-view-aware potential-field planner then steers the vehicle through a PX4-style flight controller acting on a full 6-DOF quadrotor model.

Across 160 simulated missions on unseen maps, CYCLOPS reached the goal in **90 %** of flights in every scenario. A straight-line baseline managed 0 %, a classic potential field using the same perception 60 %, and perfect perception 97.5 %. The detector reaches 0.93 precision and 0.76 recall, the learned range

error is 17 %, and one camera frame is turned into a flight command in 11 ms on a laptop CPU. The project also covers the course syllabus directly: airframe and battery modelling, flight modes and preflight checks, a stability analysis, and an endurance study.

3. Reference Paper and Objective (recap)

The project builds on Z. Zhang, Y. Zhang and Y. Cao, "*Monocular Vision-Based Obstacle Avoidance Trajectory Planning for Unmanned Aerial Vehicles*", ICUAS 2020, pp. 627–632. The paper's key idea is that avoidance must be planned around what a single camera can actually measure. A monocular camera gives direction precisely but distance only indirectly. The paper therefore defines a range-based localisation mode and an angle-based one, and studies when the latter is observable.

The objective set in the initial report was a software pipeline with these parts:

1. detect obstacles from one camera with a deep network;
2. estimate their range and bearing;
3. track obstacles, including moving ones;
4. plan a collision-free path under speed, yaw-rate and clearance constraints;
5. evaluate detection accuracy, tracking error, clearance, success rate and latency against a baseline planner.

All five parts are implemented and evaluated below.

4. System Overview

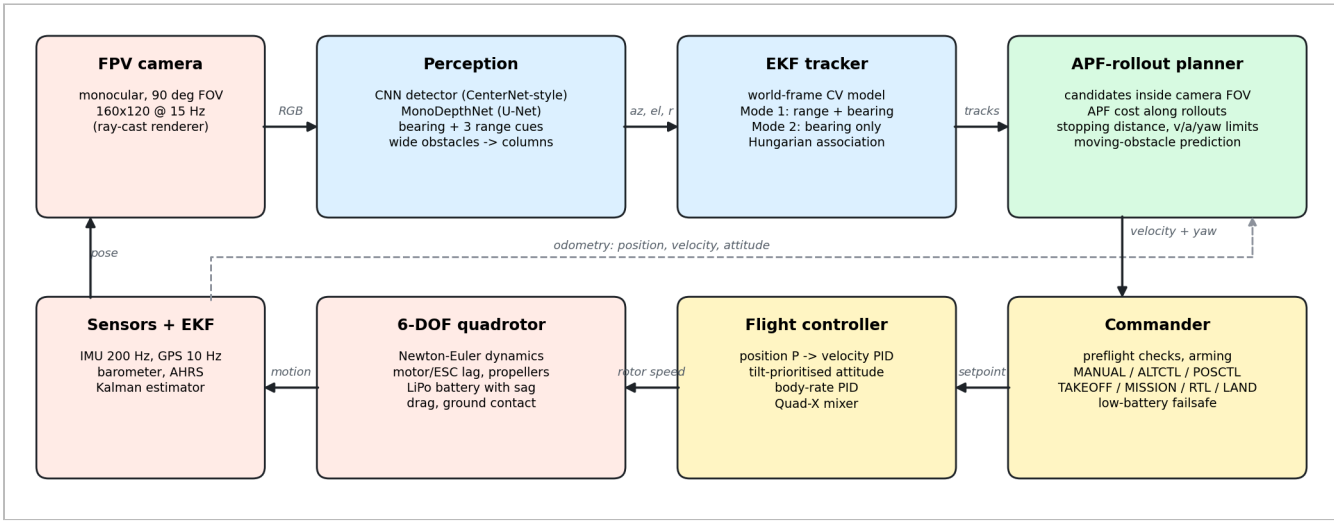


Figure 1. CYCLOPS architecture. Top row: perception and planning at 15 Hz. Bottom row: vehicle, sensors and control at 200 Hz.

The system is written in Python (NumPy, SciPy, PyTorch, Matplotlib). It is organised as independent modules that mirror the ROS 2 nodes proposed in the initial report:

- a camera producing images;
- a perception node producing detections;
- a tracker producing obstacle estimates;
- a planning node producing velocity and yaw setpoints;
- a PX4-style commander and flight controller producing motor commands.

Every component was built from scratch for this project:

- the quadrotor simulator and camera renderer;
- the synthetic dataset;
- both neural networks;
- the tracker, the planners and the live dashboard.

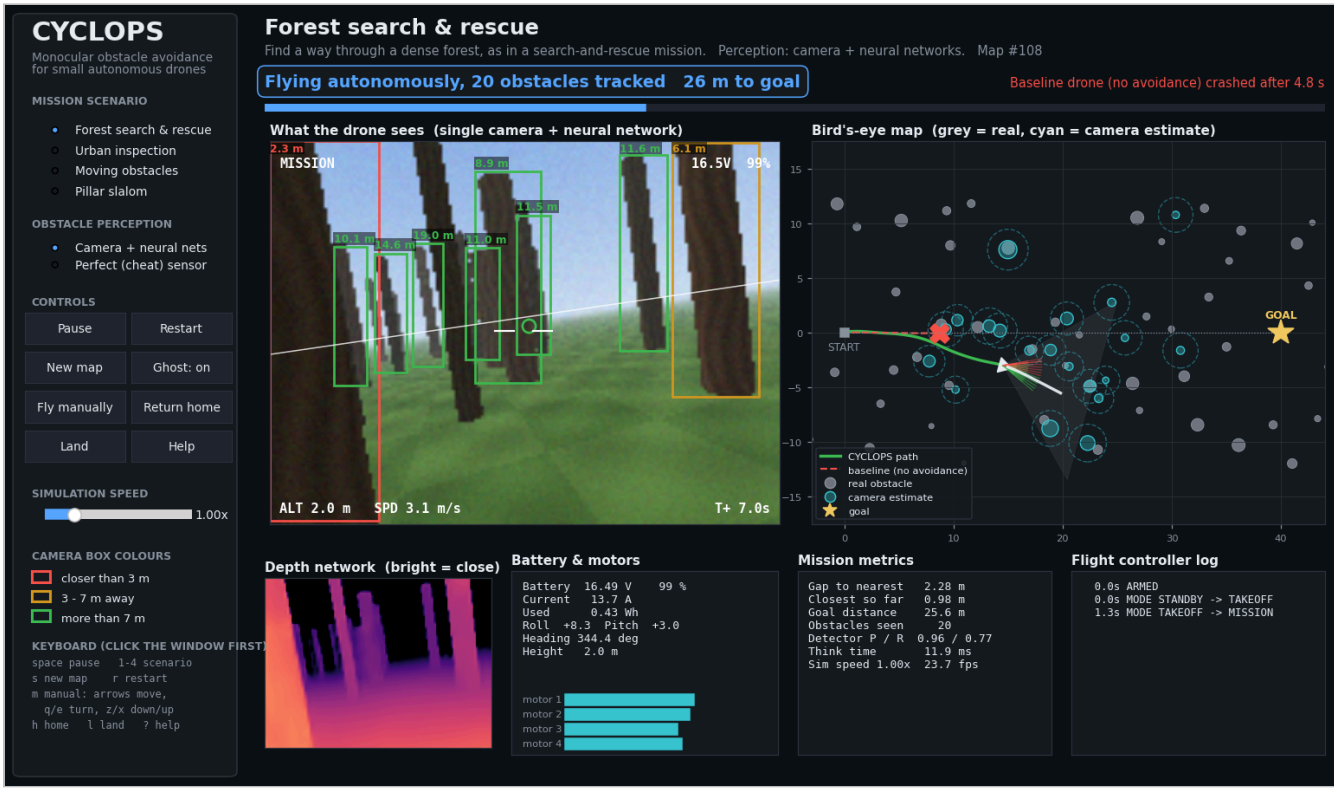


Figure 2. The live demo dashboard during a forest search-and-rescue mission. The camera view shows CNN detections with estimated distances. The map compares real obstacles (grey) with the camera's estimates (cyan), and shows the baseline drone crashing (red cross).

5. Implementation

5.1 Quadrotor model (Syllabus Modules 1 and 4)

The vehicle is a full rigid-body model, the Python equivalent of a Simulink plant block. Each propeller produces thrust $T_i = k_f \omega_i^2$ and drag torque $Q_i = k_m T_i$. Rotor speed follows the ESC command through a first-order lag.

$$m\,\dot{v} = R\,[0,\,0,\,\Sigma T_i]^T - m\,g\,e_z + F_{drag}, \qquad J\,\dot{\omega} = \tau - \omega \times J\omega, \qquad \dot{\omega}_i = (\omega_{i,cmd} - \omega_i)\,/\,\tau_m$$

The LiPo battery model includes an open-circuit-voltage curve, internal-resistance voltage sag and coulomb counting. Motor speed is limited by the present pack voltage, so the drone loses thrust as the battery drains. The IMU, GPS and barometer have realistic noise and bias, and a Kalman filter fuses them into a position and velocity estimate.

Parameter	Value	Parameter	Value
Mass (incl. battery)	1.5 kg	Airframe / arm length	Quad-X / 0.25 m
Inertia (Ixx, Iyy, Izz)	0.029, 0.029, 0.055 kg·m²	Propellers	10 inch, $k_f = 8.55\times10^{-6}$
Motor/ESC time constant	30 ms	Thrust-to-weight	2.8
Battery	4S LiPo 5000 mAh, 20 mΩ	Sensors	IMU 200 Hz, GPS 10 Hz, baro
Camera	160×120 px, 90° HFOV, 15 Hz	Physics / control rate	200 Hz

5.2 Flight controller and commander (Syllabus Modules 2 and 3)

The controller follows the PX4 multicopter cascade:

- position P;
- velocity PID;
- thrust vector and desired attitude;
- attitude P;
- body-rate PID;
- mixer with saturation handling (yaw authority is given up first).

The attitude error uses PX4's reduced-attitude method, which gives tilt priority over yaw. An early version with a plain geometric attitude error lost velocity control during fast heading changes, and this method fixed it.

A commander implements arming, a preflight checklist and flight modes. The checklist covers battery voltage and charge, IMU health, level attitude, GPS fix, estimator convergence, thrust-to-weight and camera/perception readiness. The flight modes follow PX4 naming:

- **MANUAL** (stabilise), **ALTCTL** (altitude hold) and **POSCTL** (GPS position mode), all flown with an RC-stick input model;
- **TAKEOFF**;
- **MISSION**, the intelligent obstacle-avoidance mode;
- **RTL** and **LAND**.

A low-battery failsafe triggers RTL automatically.

5.3 Monocular camera and synthetic training data (Syllabus Module 7)

The onboard camera is a vectorised ray-casting renderer (pinhole model) with lighting, fog, textures and sensor noise. Alongside the RGB image it outputs exact depth and an instance mask, so every training image is labelled automatically: no manual annotation was needed. The dataset was built in two parts:

- **Main set:** 10,000 frames (74,765 boxes) rendered from random poses in random worlds, with randomised sun, fog, exposure and colour (domain randomisation).
- **Close-range set:** 4,000 extra frames, added after the first model over-estimated short ranges.

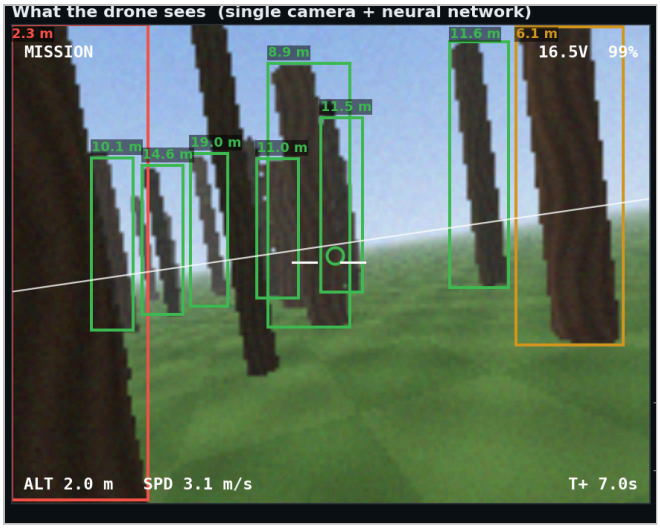


Figure 3a. Onboard view in the forest: detections coloured by distance.

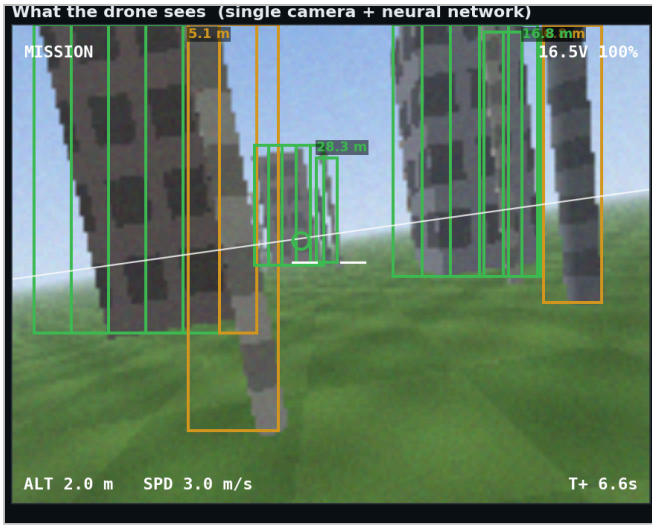


Figure 3b. Urban scene: wide buildings are split into ~1 m columns.

5.4 Deep perception networks

Network	Design	Parameters	CPU time
Obstacle detector	CenterNet-style, anchor-free: depthwise-separable encoder with a feature-pyramid decoder, and heat-map, box-size and offset heads at stride 4. Trained with focal loss.	225,637	1.4 ms
MonoDepthNet	U-Net with the same encoder, regressing bounded metric log-depth. Loss: L1 + scale-invariant + gradient matching.	264,513	2.0 ms

Training ran on the laptop GPU (Apple MPS) in two stages: 30 epochs on the main set, then a 12-epoch fine-tune with the close-range examples. Augmentation included brightness/contrast changes and horizontal flips. Both networks are small enough for a Raspberry Pi or Jetson-class companion computer.

5.5 Monocular localisation: Mode 1 and Mode 2

Bearing. For each detection, the azimuth and elevation follow exactly from the box centre and the camera intrinsics.

Range. Three monocular range cues are computed, echoing the paper's localisation modes:

1. the learned depth network, sampled inside the box;
2. flat-ground contact geometry, using the box's bottom row with the known altitude and attitude;
3. a known-size heuristic, $r = f \cdot W / w_{px}$.

Confidence. A range confidence is computed as the fraction of box pixels whose predicted depth agrees with the estimate. The tracker then uses one of two update modes:

- **Mode 1 (range + bearing)** when the range is reliable;
- **Mode 2 (bearing only)** otherwise. In Mode 2, the obstacle's position becomes observable only through the drone's own motion (parallax), exactly as analysed in the reference paper.

Wide obstacles. Obstacles wider than 2 m, such as buildings, are split into roughly 1 m image columns, each with its own learned depth (the "stixel" idea). The planner then sees their real outline and corners.

5.6 Multi-obstacle tracking

Each obstacle is tracked by an extended Kalman filter in world coordinates, with a constant-velocity model and the measurement vector [azimuth, elevation, range]. Each frame goes through these steps:

1. Detections are associated to tracks with the Hungarian algorithm, using Mahalanobis gating plus a Euclidean fallback.
2. Duplicate tracks are merged, and each obstacle's radius is estimated robustly as the median of its measurements.
3. Tracks are kept for 4 s after leaving the field of view, so obstacles beside the drone are remembered.
4. Velocity is mean-reverting: estimated motion is trusted only while the obstacle is being observed.

Moving obstacles are extrapolated 1.5–2 s ahead by the planner. Every obstacle is inflated by its estimated position uncertainty, so the avoidance is planned around what the camera really knows.

5.7 APF-rollout planner

The proposal named artificial-potential-field (APF) and vector-field-histogram (VFH) style planners. A classic APF was implemented first, but it reacts only to the current force. With a forward camera, a 0.4 s velocity response and noisy range, it oscillated between obstacles and drifted sideways into space the camera had never seen.

The final **APF-rollout** planner fixes this by testing candidate moves before committing to one:

1. Generate about 40 candidate velocities: 13 headings within ±25° of the camera axis, times 3 speeds, plus hover.
2. Predict 2 s of motion for each candidate, including the controller's lag and the obstacles' predicted motion.
3. Reject any candidate whose predicted clearance drops below 0.4 m (a stopping-distance constraint).
4. Score the rest with goal progress, the APF repulsive potential along the path, heading change, speed and smoothness, and pick the best.
5. Enforce the speed limit (4 m/s), acceleration limit (6 m/s²) and yaw-rate limit (90°/s).

Two rules handle dead ends:

- If every move is blocked, the drone hovers and yaws to look for a gap.
- If a tracked *moving* obstacle is closing in, an evasive move in any direction is allowed.

Because candidates stay inside the field of view, **the drone never flies into space it has not looked at**. This is the monocular limitation designed into the planner.

5.8 Live demo dashboard

The dashboard runs the complete loop in real time, at about 16 frames per second on a laptop. It shows:

- the onboard FPV view with detections and a goggle-style OSD (artificial horizon, flight-path marker, battery, altitude, speed);
- the depth-network output;
- a bird's-eye map of real versus estimated obstacles, the candidate paths and a "ghost" baseline drone flying the same route;
- battery and motor telemetry, mission metrics and the flight-controller log.

Clickable controls select the scenario and perception mode, pause and restart, generate a new map, switch to manual RC flight, return home and land. A Help overlay explains every panel.

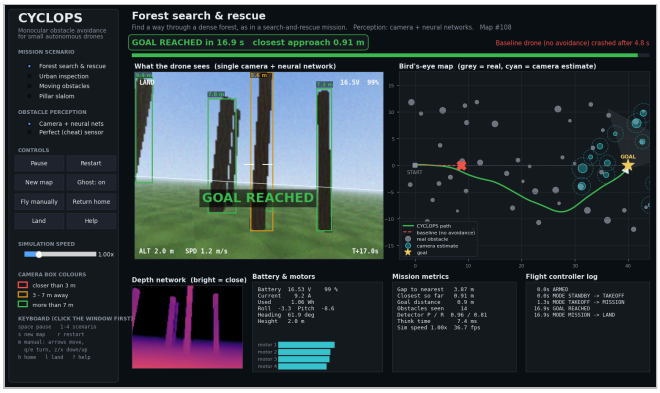


Figure 4a. Mission complete: goal reached in 16.9 s, closest approach 0.91 m; the baseline crashed after 4.8 s.

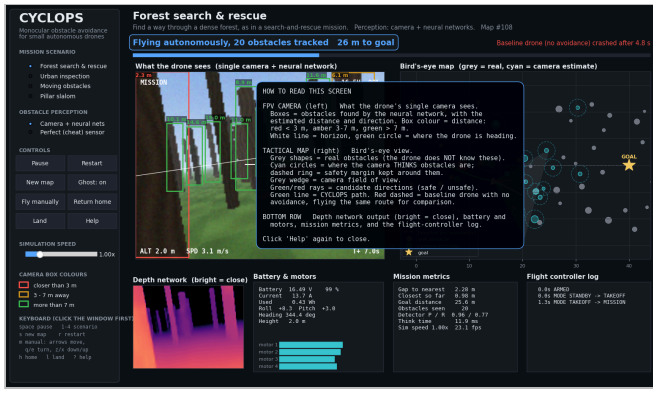


Figure 4b. Built-in Help overlay for viewers and examiners.

5.9 ROS 2 / PX4 integration

A ROS 2 package (`cyclops_ros`) wraps the same perception, tracker and planner code in two nodes:

- a **perception node**: camera image and PX4 odometry in; tracked obstacles and RViz markers out;
- a **planner node**: obstacles and odometry in; PX4 offboard velocity setpoints, arming and mode commands out.

The package also handles NED/FRD frame conversion and includes a launch file for the Gazebo `x500_mono_cam` airframe.

> The ROS 2 nodes have not yet been run against a live PX4 SITL: the development machine (macOS) had no ROS 2 installation, and only the frame conversions were unit-tested. The full evaluation in this report uses the project's own simulator. Running the nodes in PX4/Gazebo, after fine-tuning the detector on Gazebo images, is the first item of future work.

6. Experimental Setup and Metrics

Four scenarios represent the application areas of Syllabus Module 6. Each is a 40 m mission at 2 m altitude:

- **forest search-and-rescue**: about 50 trees, several on the direct path;
- **urban inspection**: buildings and poles;
- **dynamic**: spheres crossing the path at 0.6–1.4 m/s;
- **pillar slalom**.

Four methods were compared on 10 maps per scenario. The maps were never used during development, giving 160 missions in total:

1. the straight-line baseline from the proposal;
2. a classic APF with our vision pipeline;
3. **CYCLOPS**;
4. CYCLOPS with perfect ("oracle") obstacle knowledge, as an upper bound.

A mission succeeds if the drone reaches within 1 m of the goal without touching an obstacle within 60 s. The metrics are those listed in Section 8.6 of the initial report:

- detection precision and recall;

- tracking error (cross-track and altitude RMSE);
- minimum clearance;
- avoidance success rate;
- perception-to-command latency.

7. Results

7.1 Perception (1,000 held-out frames)

Metric	All obstacles (≤ 30 m)	Near obstacles (≤ 15 m)
Detection precision / recall (IoU 0.5)	0.93 / 0.76	0.92 / 0.86
F1 score / AP@0.5	0.84 / 0.76	0.89 / 0.87
Depth map: AbsRel / RMSE / δ<1.25	0.088 / 1.56 m / 0.92	

Range cue (relative error)	0–5 m	5–10 m	10–15 m	15–20 m	20–30 m	Overall
Learned depth network (Mode 1)	**22.7 %**	**15.5 %**	**15.3 %**	**18.5 %**	**17.3 %**	**17.2 %**
Flat-ground contact geometry	87.6 %	53.3 %	21.0 %	24.1 %	16.7 %	35.2 %
Known-size heuristic	54.3 %	45.3 %	44.6 %	46.2 %	48.9 %	47.3 %

The learned depth is the only cue that stays accurate at close range, where it matters most. The ground-contact cue fails up close because the bottom of a nearby obstacle leaves the image.

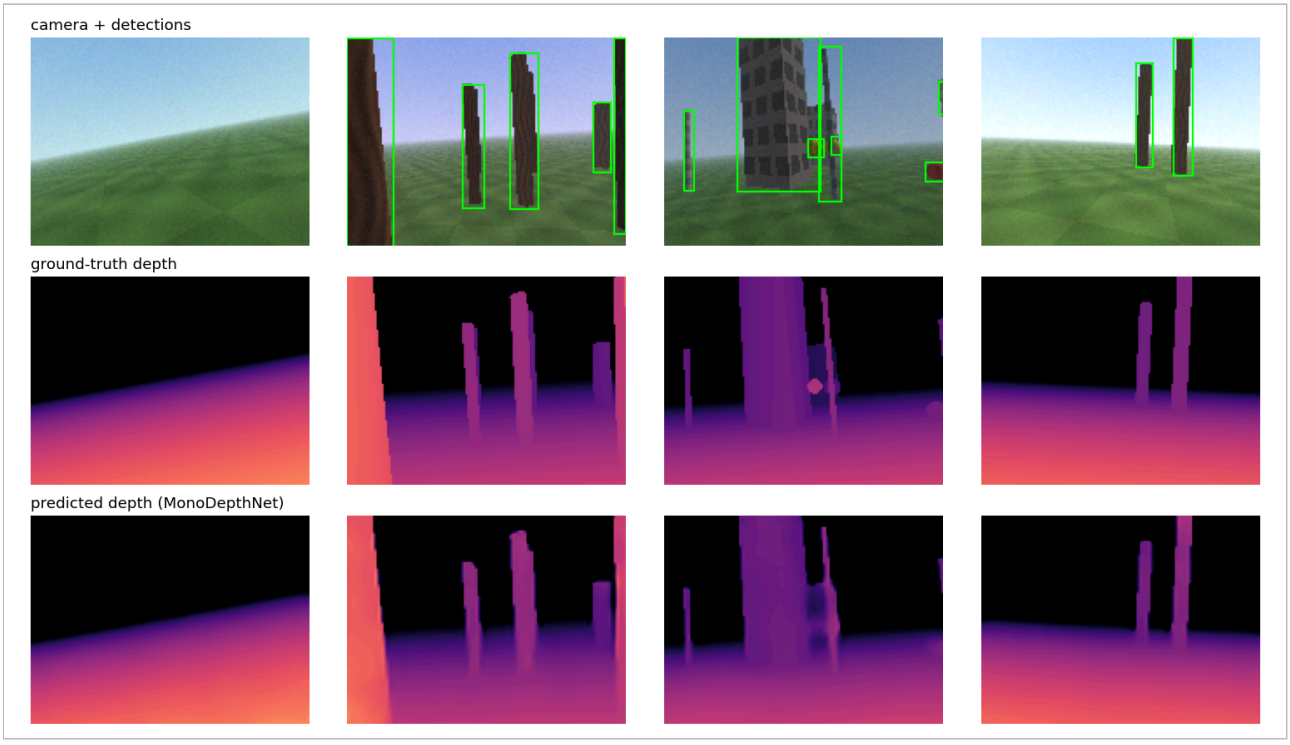


Figure 5. Validation examples. Row 1: camera image with detections. Row 2: ground-truth depth. Row 3: depth predicted from the single image.

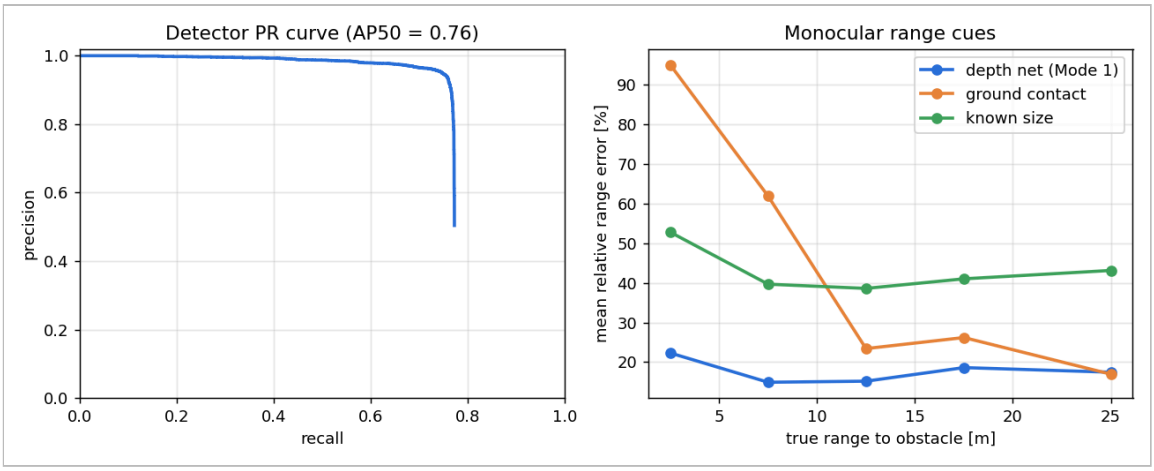


Figure 6. Detector precision-recall curve (left) and range error of the three monocular cues against distance (right).

7.2 Obstacle-avoidance benchmark (160 missions)

Method	Success	Collision	Timeout	Mean min. clearance	Mission time	Energy
Baseline (straight line, no avoidance)	0 %	100 %	0 %	–	–	–
Classic APF + our vision	60 %	40 %	0 %	0.40 m	18.1 s	1.24 Wh
CYCLOPS (APF-rollout + vision)	**90 %**	**7.5 %**	**2.5 %**	**0.68 m**	**25.4 s**	**1.66 Wh**
Oracle perception (upper bound)	97.5 %	0 %	2.5 %	0.71 m	20.8 s	1.36 Wh

Success rate per scenario	Forest SAR	Urban	Dynamic	Pillars
Baseline	0 %	0 %	0 %	0 %
Classic APF + vision	40 %	70 %	50 %	80 %
CYCLOPS	**90 %**	**90 %**	**90 %**	**90 %**
Oracle perception	100 %	90 %	100 %	100 %

The gap between CYCLOPS and the oracle is the cost of seeing with one camera: only 7.5 percentage points. The gap between CYCLOPS and the classic APF, which uses identical perception, is the benefit of the rollout planner: 30 points. CYCLOPS takes longer missions (25 s against 21 s for the oracle) because it slows down and re-plans when distance estimates are uncertain, a deliberate trade of speed for safety.

Altitude was held to 4 cm RMSE. The cross-track deviation from the straight line (3.9 m) reflects the detours needed around obstacles, not tracking error.

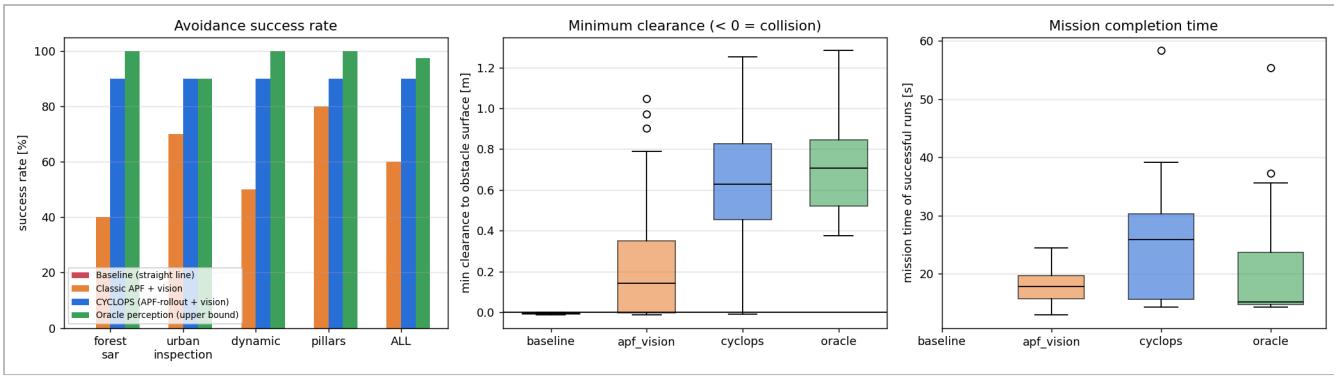


Figure 7. Success rate per scenario, minimum clearance distribution (below 0 = collision), and mission time.

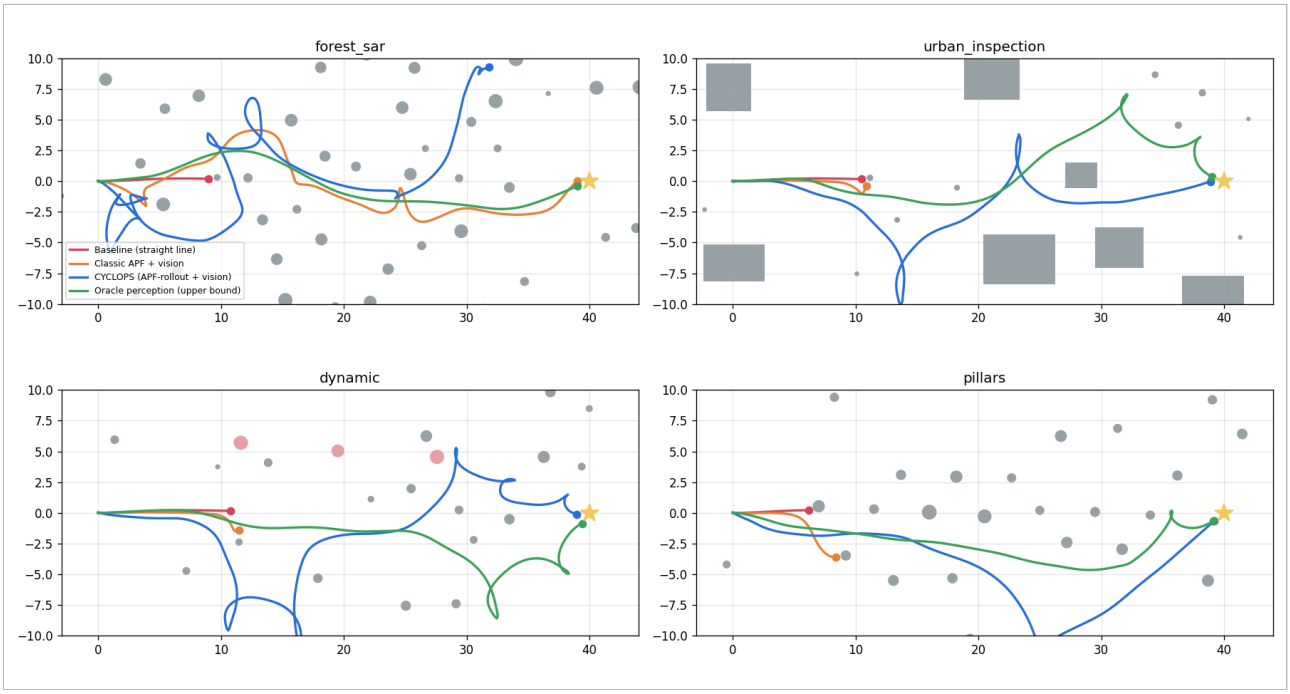


Figure 8. Paths flown by each method on the same map in each scenario (grey = obstacles, pink = moving obstacles).

7.3 Real-time performance

Network inference takes 3.4 ms per frame on the CPU (detector 1.4 ms + depth 2.0 ms). The complete chain from camera frame to velocity command averages **11.4 ms**, with a 95th percentile of 23.6 ms. This covers perception, localisation, tracking and planning, and is well inside the 66 ms budget of a 15 Hz camera.

8. Stability and Control Analysis (Syllabus Module 5)

Open loop. Linearising the 6-DOF model at hover gives six eigenvalues at exactly zero, with the others only slightly negative (from drag). The bare airframe is therefore *neutrally* stable: there is no natural restoring moment in roll, pitch or yaw. All static and dynamic stability must come from the feedback controller.

Closed loop. The full rank of the controllability matrix confirms the four rotors can control all 12 states. The tables and figures below give the closed-loop results.

Axis / test	Result
Roll (lateral) and pitch (longitudinal) 10° step	rise time 0.23 s, overshoot 0.2 %, 5 % settling 0.33 s
Yaw (directional) 45° step	rise time 0.67 s, overshoot 1.7 %, settling 1.08 s
Altitude +1 m step	rise time 1.31 s, no overshoot, settling 1.86 s
Gust: 6 N horizontal push for 0.5 s	peak deviation 0.74 m, back within 10 cm after 3.5 s
Roll rate loop / attitude loop	phase margin 72° / 68°, gain margin ∞ / 18.5 dB, crossover 14.7 / 6.0 rad/s
Yaw rate loop / attitude loop	phase margin 78° / 67°, gain margin ∞ / 22.2 dB
Stability boundary	roll loop unstable only for attitude gain > 50.6 (design value 6: 8.4× margin)

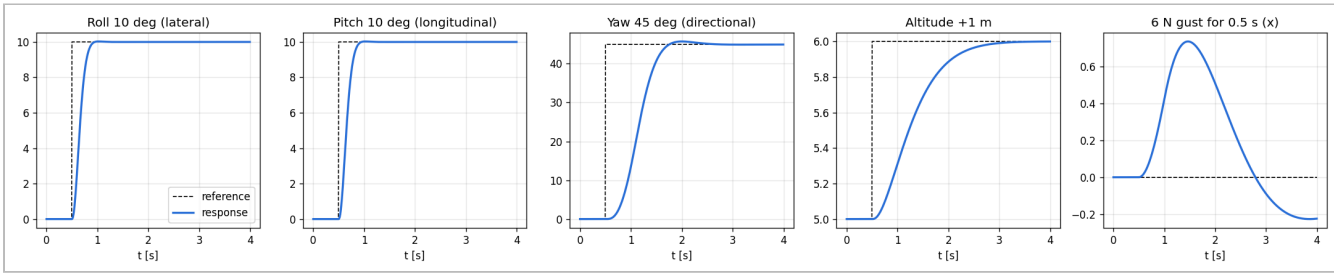


Figure 9. Closed-loop responses on the nonlinear model: roll, pitch, yaw, altitude and gust rejection.

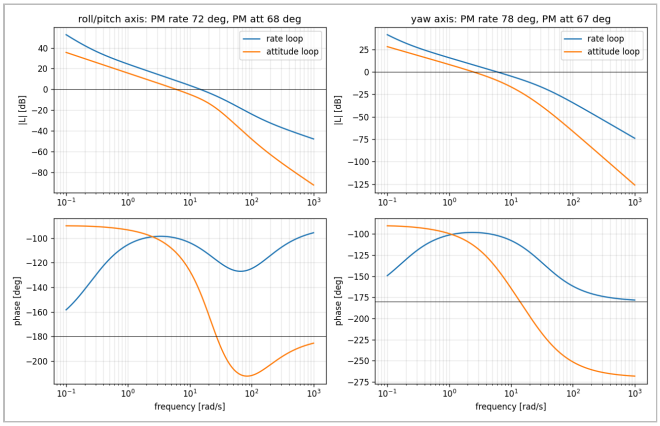


Figure 10a. Bode plots of the rate and attitude loops.

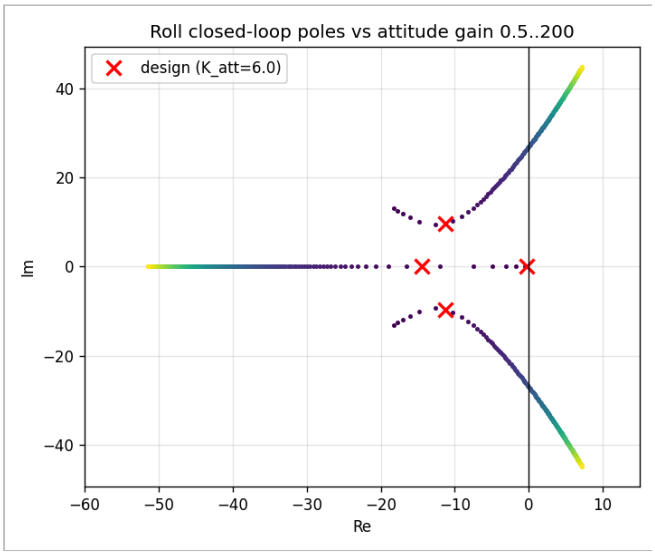


Figure 10b. Closed-loop roll poles as the attitude gain varies from 0.5 to 200.

9. Endurance Study (Syllabus Modules 1 and 7)

The 10-inch propellers give a disc loading of 72.6 N/m². Hover power is 80 W in ideal momentum theory, 178 W with a figure of merit of 0.65 and losses, and 214 W measured on the simulator.

Forward-flight momentum theory, with Glauert's induced velocity and parasite drag, shows the classic "bucket" curve:

- **best endurance** at 6.8 m/s: 23.5 min, compared with 20 min in hover;
- **best range** at 11.3 m/s: 12.8 km on 59 Wh of usable energy.

Bigger batteries show diminishing returns: doubling capacity from 5 to 10 Ah raises hover time by only about 33 %, because the extra cells must also be lifted.

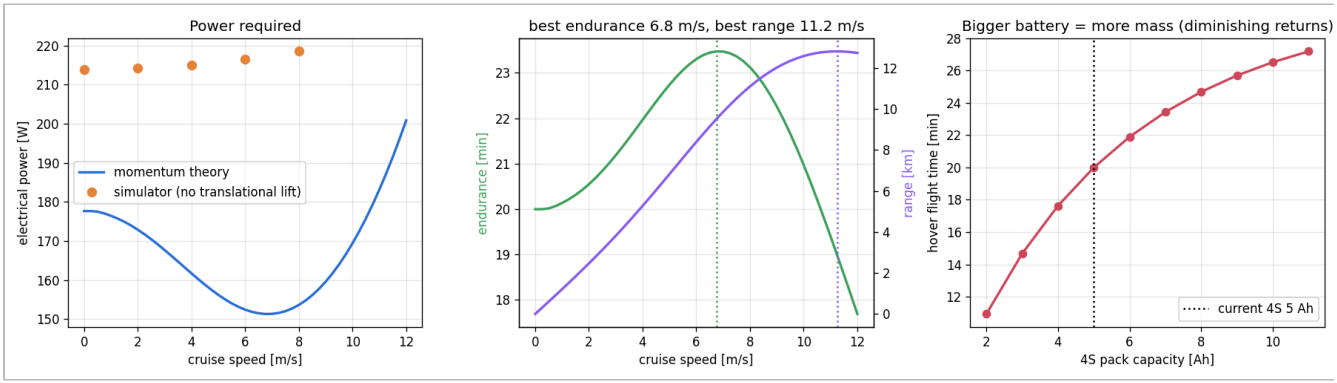


Figure 11. Power required against speed, endurance and range against speed, and hover time against battery capacity.

10. Mapping to the BCSE428L Syllabus

Module	Where it appears in CYCLOPS
1 – Introduction (types, airframe, batteries, motors, ESC, propellers)	Quad-X airframe, propeller thrust/torque model, motor/ESC lag, LiPo battery with voltage sag (Sec. 5.1, 9)
2 – Design fundamentals (flight controllers, RC, FPV, telemetry)	PX4-style cascaded flight controller, RC stick model with manual flight, FPV view with OSD, telemetry panel (Sec. 5.2, 5.8)
3 – Drone basics (preflight, flight modes, roll/pitch/yaw, GPS autopilot, intelligent modes)	Preflight checklist, MANUAL/ALTCTL/POSCTL/TAKEOFF/MISSION/RTL/LAND, low-battery failsafe (Sec. 5.2)
4 – Modelling and control	6-DOF Newton–Euler model, sensor models and Kalman filter (Sec. 5.1)
5 – Stability and control	Open-loop eigenvalues, Bode margins, root locus, roll/pitch/yaw step responses, gust rejection (Sec. 8)
6 – Applications	Search-and-rescue forest, urban infrastructure inspection, dynamic obstacles, slalom (Sec. 6)
7 – Expanding drone abilities (camera, FPV, sensors, speed, flight time)	Monocular camera with deep perception; speed and battery-capacity endurance study (Sec. 5.3–5.5, 9)

11. Challenges Faced and How They Were Solved

- **Over-estimated distance at close range.** The first depth network over-estimated distance by 31 % below 1.5 m, which caused late reactions and collisions. Adding 4,000 close-range training frames (hard-example mining) and fine-tuning reduced the bias to +9 %.
- **Flying sideways into unseen space.** The classic APF commanded motion outside the camera's view. Restricting candidate motions to a cone inside the field of view, and letting yaw lead into turns, fixed this.

- **Lost velocity control during fast yaw.** A geometric attitude error mixed large yaw errors into roll and pitch. It was replaced by PX4's tilt-prioritised reduced-attitude control.
- **Drifting and duplicate obstacle tracks.** These were fixed with velocity mean-reversion, a Euclidean association fallback, duplicate merging and median radius estimation.
- **Buildings clipped at their corners.** Fitting one circle to a building missed its corners. Splitting wide detections into columns, each with its own learned depth, recovered the real outline.

12. Limitations and Future Work

- **Blind spots.** Obstacles approaching from outside the 90° field of view cannot be seen; one dynamic-scenario collision was of this kind. A wider lens or a rear camera would help.
- **Synthetic training data only.** The networks were trained on synthetic images. Moving to Gazebo or real footage needs fine-tuning, which the training pipeline already supports.
- **No live PX4 test yet.** The ROS 2/PX4 nodes are written but not yet tested in SITL (Sec. 5.9).
- **Observability analysis.** As stated in the initial report, a formal observability analysis of Mode 2, with mode switching driven by geometry rather than depth confidence, remains future work, as does flight on a real low-cost quadrotor.

13. How to Run

```
git clone https://github.com/ac265640/CYCLOPS.git
cd CYCLOPS
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/live_demo.py                 # live demo (trained weights included)
python scripts/benchmark.py --seeds 10      # reproduce the benchmark
python scripts/stability_analysis.py        # Module 5 analysis
python scripts/endurance_analysis.py        # Modules 1 and 7 analysis
python -m pytest -q                         # 8 unit / integration tests
```

14. Conclusion

The project set out to show that a quadrotor can avoid obstacles using only one camera, if the avoidance is planned around what that camera can actually measure. CYCLOPS implements this idea end to end:

- deep networks for detection and depth;
- the reference paper's Mode 1 and Mode 2 localisation inside an EKF tracker;
- a planner that keeps motion inside the camera's field of view and accounts for estimate uncertainty and moving obstacles;
- a realistic, course-aligned flight stack.

With 90 % mission success against 0 % for the baseline, within 7.5 points of perfect perception, and 11 ms latency, the pipeline meets the objectives of the initial report. The code, results, videos and this report are publicly available at the links in Section 1.

References

1. Z. Zhang, Y. Zhang, and Y. Cao, "Monocular Vision-Based Obstacle Avoidance Trajectory Planning for Unmanned Aerial Vehicles," in *Proc. ICUAS*, Athens, 2020, pp. 627–632. doi:10.1109/ICUAS48674.2020.9213901
2. X. Zhou, D. Wang, and P. Krähenbühl, "Objects as Points," 2019. arXiv:1904.07850
3. R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer" (MiDaS), 2019. arXiv:1907.01341
4. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 2015. arXiv:1505.04597
5. D. Eigen, C. Puhrsch, and R. Fergus, "Depth Map Prediction from a Single Image using a Multi-Scale Deep Network," 2014. arXiv:1406.2283
6. T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," 2017. arXiv:1708.02002
7. A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. arXiv:1704.04861
8. O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," *Int. J. Robotics Research*, vol. 5, no. 1, 1986. doi:10.1177/027836498600500106
9. D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, 1997. doi:10.1109/100.580977
10. J. Borenstein and Y. Koren, "The Vector Field Histogram – Fast Obstacle Avoidance for Mobile Robots," *IEEE Trans. Robotics and Automation*, 1991. doi:10.1109/70.88137
11. H. W. Kuhn, "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, 1955. doi:10.1002/nav.3800020109
12. H. Badino, U. Franke, and D. Pfeiffer, "The Stixel World – A Compact Medium Level Representation of the 3D-World," DAGM, 2009.
13. PX4 Autopilot documentation: ROS 2 user guide, docs.px4.io/main/en/ros2/user_guide.html; controller diagrams, docs.px4.io/main/en/flight_stack/controller_diagrams.html
14. PyTorch, pytorch.org · ROS 2, docs.ros.org · Gazebo, gazebosim.org